Enunciado 5

1. Análisis de código **(6 pts.)**

Dado el siguiente código en Flask:

```python
@bp.route('/buscar', methods=['POST'])
def buscar():
    criterio = request.form['query']

    resultados = db.session.scalars(
        select(LugarHistorico).where(LugarHistorico.nombre.icontains(criterio))
    ).all()

    session['ultimos_resultados'] = [r.to_dict() for r in resultados]

    return redirect('/resultados')
```

a) **Explicá** qué intenta realizar esta función dentro del ciclo Request/Response.
b) **Identificá dos problemas** principales (obligatoriamente al menos uno sobre uso inadecuado de POST y guardado masivo en sesión para búsquedas y otro de URLs hardcodeadas en redirecciones) **y explicá técnicamente** por qué representan un fallo y cuál es la solución adecuada (no tenés que escribir el código corregido, solo explicar la solución verbalmente).
c) **Respondé:** ¿por qué las operaciones de búsqueda y filtrado deben realizarse mediante el método HTTP GET con parámetros en la URL en lugar de utilizar peticiones POST y guardar los resultados en la sesión?

2. Aplicación en el Trabajo Integrador **(6 pts.)**

Analizá el uso del objeto *session* en la arquitectura de tu trabajo integrador. **Explicá verbalmente:**

- ¿En qué módulo se gestionan las variables de sesión y por qué es una mala práctica almacenar colecciones de datos o grandes volúmenes de información en la sesión?
- ¿Qué diferencia existe entre almacenar estado en la sesión en el cliente (cookies firmadas por defecto) versus utilizar almacenamiento de sesión en el servidor (Server-Side Session)?

3. Demostración en vivo **(3 pts.)**

Mostrá tu aplicación corriendo **localmente**. Realizá una operación que modifique el estado de la sesión (por ejemplo:. cambiar de rol o iniciar sesión) y mostrá en las herramientas del navegador cómo impacta esa modificación en la cookie.

1) a) ¿Qué intenta realizar esta función dentro del ciclo Request/Response?

La función define una ruta /buscar que acepta peticiones HTTP POST. Cuando recibe una petición, obtiene el criterio de búsqueda enviado mediante el formulario, consulta la base de datos para encontrar los lugares históricos cuyo nombre contenga ese criterio y convierte los resultados en diccionarios. Después, guarda esos resultados en la sesión del usuario, bajo la clave ultimos_resultados, y finalmente redirige al usuario a la ruta /resultados. Por lo tanto, dentro del ciclo Request/Response, la función procesa una petición, accede a la base de datos, modifica el estado de la sesión y devuelve una respuesta de redirección.

b) ¿Cuáles son los problemas principales y cómo se solucionarían?

1. Uso inadecuado de POST
Una búsqueda normalmente es una operación de lectura que no modifica los recursos del servidor. Por eso, sería más apropiado utilizar GET y enviar el criterio de búsqueda como parámetro de la URL. Esto permite compartir la búsqueda, guardarla en favoritos y recuperarla mediante la URL.

2. Guardado masivo de resultados en la sesión
Guardar todos los resultados de la búsqueda en la sesión es una mala práctica porque la sesión debería almacenar información pequeña y necesaria para mantener el estado del usuario. Si se utiliza la sesión predeterminada de Flask, los datos se guardan en una cookie firmada, por lo que almacenar muchos resultados aumenta su tamaño y puede superar los límites del navegador. Además, la información puede quedar desactualizada respecto de la base de datos. La solución es guardar únicamente el criterio de búsqueda en la URL y volver a consultar la base de datos cuando se acceda a la ruta de resultados.

c) ¿Por qué la búsqueda debería realizarse con GET y no con POST y guardarse en la sesión?

Porque una búsqueda es una operación de consulta que, en condiciones normales, no modifica el estado de los recursos del servidor. GET permite representar el criterio de búsqueda en la URL, por lo que la consulta se puede compartir, guardar en favoritos, recuperar y navegar mediante el historial del navegador. Además, evita utilizar la sesión como almacenamiento de resultados. En su lugar, el servidor recibe el criterio mediante GET, consulta la base de datos y devuelve los resultados correspondientes. Esto también simplifica el flujo de navegación y evita depender de datos almacenados previamente en la sesión.

2) Aplicación en el trabajo integrador

En mi trabajo integrador la sesión se gestiona en el controller de login: al autenticar guardo una sola clave, el user_id, y en el logout la limpio. El rol y los permisos los consulto en la base en cada request, así nunca quedan desactualizados. Guardar colecciones grandes en la sesión es mala práctica porque cada request tiene que leerlas y escribirlas completas: consume recursos del servidor, escala mal y los datos pueden quedar desactualizados respecto de la base. Por eso guardo solo un identificador. Ahora, sobre dónde se guarda la sesión: en el cliente, todo viaja dentro de la cookie, que va firmada pero no cifrada, tiene un límite de unos 4 KB y no se puede invalidar desde el servidor. En mi proyecto uso sesión del servidor con Flask-Session: los datos quedan en archivos en el servidor y la cookie lleva solo un ID aleatorio. Así no se expone información al navegador y puedo revocar sesiones cuando haga falta.